ELI Perception — Vision, Voice, Wake Word, Tone, OS Control

Contents

Files 1
Vision (vision.py + ambient_vision.py + analyze_image.py) 2
Speech-to-text (audio_stt.py, local_whisper_stt.py) 2
Wake word (wakeword.py, 2026-06-08) — self-trained, robust over music 2
Voice profile + tone (voice_profile.py, 2026-06-08) — foundation for emotion 2
Text-to-speech (tts_router.py) 3
OS control (os_controller.py) 3
File analysers 3
Honest assessment 3
Update — 2026-06-09 (VRAM-aware STT; drag-drop keeps the full path) 3

eli/perception/ (~20 files). ELI's senses and hands: local vision, speech-to-text, text-to-speech, a **self-trained wake-word detector**, a **voice-profile/tone** subsystem, and OS control. All local, no APIs, no third-party accounts.

Files

File	LOC	Role
audio_stt.py	~1.6k	STT + mic capture + ducking + adaptive pause + wake/voice capture
wakeword.py	~450	**self-trained, music-robust wake-word detector** (openWakeWord features + custom head)
voice_profile.py	~420	**prosody + labelled-emotion** (tone/question detection)
tts_router.py	667	
vision.py	692	
os_controller.py	573	
screen_locator.py	389	locate UI elements on screen
gaze_engine.py	298	
log_rotation.py	215	log housekeeping
analyze_pdfs/image/mesh/csv.py	~600	file-type analysers
ambient_vision.py	207	periodic screen glances (off by default)
local_whisper_stt.py	257	
voice_worker(_streaming).py, eli_listen.py, extract_equations.py	small	workers/helpers

Vision (`vision.py` + `ambient_vision.py` + `analyze_image.py`)

The working local-vision stack (see memory `eli-image-analysis`): - VL model (default Qwen2.5-VL, configurable) loaded via `Qwen25VLChatHandler`; **CPU clip forced** by monkeypatching `mtmd_cpp.mtmd_init_from_fi` — the GPU mtmd clip path **segfaults** on compute-7.5 cards. - **Hot-swap**: unload text model → load VL → infer → restore in `finally`, holding `gguf_inference._LLM_CALL_LOCK`. - **Co-resident fast path**: a small model (Moondream2 Q4) loaded first, with the text model's ctx capped (`vision_coresident_text_ctx`) so both fit 8GB; `prefer_fast=True` uses it without a swap. - Images downscaled to `vision_max_image_px` (1280) to avoid context overflow; `repeat_penalty`/`top_k`/`top_p` set to kill a repetition loop. - `ambient_vision.py`: optional periodic screen glances (OFF by default). A guarded daemon re-reads the toggle/interval each cycle and **skips a glance whenever the shared LLM lock is busy** (so it never steals the model mid-reply). Stores a short description as memory for rolling awareness.

Speech-to-text (`audio_stt.py`, `local_whisper_stt.py`)

A large, feature-dense STT module: - Mic capture + Whisper transcription (`local_whisper_stt`). - **Output ducking** — lowers system sink volume while listening (`_eli_duck_output`/`_eli_restore_output` via `wpctl`) so ELI doesn't hear itself. - **Echo/self-hearing suppression** (`_eli_echo_like_assistant_output`, `_eli_media_probably_audible`). - Transcript cleanup: `_cleanup`, `_collapse_repeated_phrase`, `_eli_fast_command_alias` (maps spoken phrases to commands), `_is_safe_direct`/`_allow_direct_chat` (gates which utterances bypass to chat). - ALSA stderr suppression to keep the console clean. - **Duration-adaptive end-of-phrase pause** (`_listen_adaptive_pause`, 2026-06-08) — stock `sr.listen()` reads `pause_threshold` once per phrase, so one value can't be both snappy for commands and tolerant of long dictation. A faithful copy of sr's capture with a single dynamic condition: short commands finalise after `ELI_STT_SHORT_PAUSE` (0.5s) of silence; a prompt speaking past `ELI_STT_LONG_AFTER` (12s) needs `ELI_STT_LONG_PAUSE` (2s), so a mid-sentence pause no longer cuts it. Flag-gated (`ELI_STT_ADAPTIVE_PAUSE`) with fallback to stock `listen()`. - **Generic mic-capture script** — one mechanism (no second microphone) drives both wake-word enrollment and voice/emotion training: a list of `{prompt, sink}` steps, each captured (past the TTS-echo guards) clip routed to its sink, the next cue spoken, a done callback after the last. `begin_capture_script` / `begin_wake_enrollment` / `begin_voice_training`. - **Acoustic wake hook** — when unarmed and a wake model is trained, the captured audio is scored by `wakeword` and, if it fires, the wake word is injected so the existing `VoiceGate` arms — catching the wake word even when whisper transcribed the music. **Per-turn tone hook** — on a real command, `voice_profile.classify_tone` is run and published on a side-channel for cognition (below).

Wake word (`wakeword.py`, 2026-06-08) — self-trained, robust over music

Transcription-based wake matching can't hear "computer" over loud music (whisper transcribes the music). Instead ELI **trains its own** detector, 100% locally: - **Positives**: the wake phrase synthesised by ELI's OWN **Piper TTS** across several voices and speeds. - **Augmentation (the robustness)**: each positive is mixed with noise/music at random SNRs, so the classifier learns to spot the wake word *through* a music bed. - **Features**: openWakeWord's open, bundled melspectrogram→embedding extractor (no account, no download barrier). - **Custom head**: a small torch classifier ELI owns; `train_model()` → `models/wakeword/eli_wake_head.pt` (gitignored). `WakeDetector.score_audio`/`is_wake` slide a 1.5s window. Validated: clean wake = 1.00, wake+loud music (3 dB) = 1.00, hard-negative/music-only = 0.00. - **User-settable phrase** — `get`/`set_wake_phrases` persists any phrase ("change the wake word to athena"); it feeds both this model and the transcription matcher. - **Personalisation** — enrolled real-mic clips of the user are folded in as heavily- weighted positives. - Actions: `WAKE_SET` / `WAKE_TRAIN` / `WAKE_ENROLL`. Fully fallback-safe (`ELI_WAKE_ACOUSTIC=0`; no model → the transcription matcher).

Voice profile + tone (`voice_profile.py`, 2026-06-08) — foundation for emotion

Deliberately SEPARATE from the wake word — this is *how* the user speaks, the basis for tone/emotion and question-vs-statement: - **Prosody (real, numpy only)**: per-clip autocorrelation **F0/pitch** track, energy, voiced ratio, speaking rate, and a **terminal-pitch slope** (`analyze_prosody`). - **Question**

vs statement (working): rising terminal pitch ⇒ question (`question_or_statement`). - **Labelled emotion**: `add_labelled_sample` + a nearest-centroid classifier (`train_emotion_classifier`) over neutral/happy/angry/excited/sad — robust for the small data a quick enrollment yields, upgradeable to a NN without touching callers. - **`classify_tone`** returns emotion + confidence, arousal, and the question/statement cue. `build_profile` learns the user's baseline so tone is scored *relative* to how they normally sound. - **`TRAIN_VOICE`** runs one unified guided session (wake reps + the same line said in each emotion); modules stay separate, the user does one flow. - **Wired into cognition**: STT publishes the per-turn read on a fresh, timestamped side-channel (`set_last_tone`); `engine._build_enhanced_system` reads `get_last_tone` and prepends a speaker-voice cue so ELI adapts its delivery (warmer if upset, brisk if energetic). `ELI_VOICE_TONE=0` disables. Complementary to the text-based `cognition/tone_analyzer` (persona preferences), not a duplicate.

Text-to-speech (`tts_router.py`)

Multi-backend router with graceful fallback: **Piper** (packaged binary under `tts_piper/` or `models/tts/piper`, voice-dir resolution) → **espeak-ng / espeak**. Resolves voices from several candidate dirs so a packaged build or a source checkout both work.

OS control (`os_controller.py`)

Platform-aware (Linux/Windows/macOS) screenshot, volume, keyboard, clipboard. `take_screenshot(region)` picks the right tool per platform (`gnome-screenshot`, PIL `ImageGrab`, platform CLIs); fails gracefully where a capability isn't available (e.g. area screenshots on some Windows installs). `screen_locator.py` finds UI targets for click/automation.

File analysers

`analyze_pdfs`, `analyze_image`, `analyze_csv`, `analyze_mesh`, `extract_equations` — typed handlers behind the `ANALYZE_*` actions, feeding extracted content (text/OCR/structure) into the grounded pipeline.

Honest assessment

- **Strong:** this is the layer that makes ELI genuinely multimodal and local — eyes (VL + OCR + ambient), ears (Whisper + ducking), mouth (Piper/espeak), hands (cross-platform OS control). The vision co-residence + hot-swap + CPU-clip workaround is real engineering against hard 8GB-GPU constraints.
- **Weak / watch:**
  1. **Vision latency** — the hot-swap unloads/reloads the text model per glance; even the co-resident path runs CPU clip (~3.5s). Acceptable, not fast.
  2. `audio_stt.py` is a **1.48k-line** module mixing capture, ducking, echo suppression, command aliasing, and cleanup — wants splitting.
  3. **Residual "7B" comment** in `ambient_vision.py` ("the 7B vision model hot-swaps…") — cosmetic, but inconsistent with the model-agnostic line; should read "the vision model". (Flagged for a future scrub.)
  4. `gaze_engine.py` is experimental and off by default — fine, but it's carried weight.
  5. Platform tools are detected at call time; a machine missing `gnome-screenshot`/`wpctl`/`piper` degrades silently to fallbacks (good) but there's no single "what perception capabilities do I actually have here?" probe surfaced to the user.

Update — 2026-06-09 (VRAM-aware STT; drag-drop keeps the full path)

- **STT is VRAM-aware** (`local_whisper_stt`). faster-whisper defaults to GPU only when the card is big enough for whisper AND a typical main model (≥12 GB total VRAM), else CPU — so on an 8 GB card whisper no longer preloads ~2 GB and starves the main model (observed: free_vram 4083

MB → gpu_layers=11; after the fix free_vram 7092 MB → gpu_layers=99). `small.en` int8 on CPU is ~1–2 s for a short command. Explicit `ELI_WHISPER_DEVICE` still wins; `ELI_WHISPER_GPU_MIN_MB` tunes the threshold.

- **Drag-and-drop keeps the FULL path.** A file dropped into the chat input now expands to `[File: <full path>]` followed by the inlined content (was content + basename only), so "fix/examine/edit this file" can resolve the file on disk while "summarise this" still has the content. PDFs keep their path too; combined with the FIX_FILE last-file memory, a bare follow-up "fix it" recovers the file named a turn earlier.